# Verification and Validation Report: Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

April 6, 2026

# 1 Revision History

| Name(s) | Date | Notes |
|---|---|---|
| Maham, Vaisnavi, Eman, Cieran, Matthew | March 9th, 2026 | Created inital version of VnV Report |
| Vaisnavi S. | April 4th, 2026 | Applied feedback to the VnV Report |
| Vaisnavi S. | April 5th, 2026 | Applied feedback to the VnV Report |
| Vaisnavi S. | April 6th, 2026 | Finalized the final version of the VnV Report |

# Contents

# List of Tables



# List of Figures

# 2 Symbols, Abbreviations and Acronyms

| symbol | description |
|---|---|
| VnV | Verification and Validation |
| SRS | Software Requirements Specification |
| API | Application Programming Interface |
| TC | Test Case |
| UT | Usability Test |
| PR | Performance Test |
| RFT | Reliability and Fault-Tolerance Test |
| SR | Security Test |
| MS | Maintainability and Support Test |
| FR | Functional Requirement |
| SR-AR | Security Access Requirement |

# 3 Introduction

This document presents the results of all tests conducted to evaluate the behaviour and output of PhysioCompanion. The report aligns with the test cases outlined in the Verification and Validation (VnV) plan. It begins with an evaluation of both functional and non-functional requirements, followed by individual module or functionality unit tests . The changes implemented after usability testing are then discussed, and the document concludes with code coverage statistics and analysis.

# 4 Functional Requirements Evaluation

1. **test-TC1 : Physiotherapist Registration (Valid License)**

   **Initial State:** App launched; registration screen accessible; Firebase Auth and Firestore active.

   **Input:** `registerPhysio()` called with a valid license format and an active license in `validLicenses`.

   **Output:** Auth user created. Firestore `users/{uid}` written with `role="physio"`, `verified=true`, and `createdAt`. All assertions passed.

   **Result:** Pass

2. **test-TC2 : Physiotherapist Registration (Invalid License Format)**

   **Initial State:** App launched; registration screen accessible.

   **Input:** `registerPhysio()` called with an invalid license format (missing dash or incorrect digits).

   **Output:** Error "Invalid license format" thrown. No Auth user or Firestore writes created.

   **Result:** Pass

3. **test-TC3 : Physiotherapist Registration (Unverified License)**

   **Initial State:** App launched; registration screen accessible.

   **Input:** `registerPhysio()` called with valid format but license not found or `active=false` in `validLicenses`.

**Output:** Error "License not verified" thrown. No Auth user or Firestore writes created.

**Result:** Pass

4. **test-TC4 : Patient Registration (Valid Invite Code)**

   **Initial State:** App launched; `inviteCodes` collection seeded with an active, unused code containing a `physioId`.

   **Input:** `registerPatient()` called with a valid invite code.

   **Output:** Auth user created. Firestore `users/{uid}` written with `role="patient"` and stored `physioId`. Invite code updated with `used=true` and `usedBy={uid}`.

   **Result:** Pass

5. **test-TC5 : Patient Registration (Non-Existent Invite Code)**

   **Initial State:** App launched; `inviteCodes` collection does not contain the submitted code.

   **Input:** `registerPatient()` called with a code that does not exist in `inviteCodes`.

   **Output:** Error "Invalid invite code" thrown. No Auth user or Firestore writes created.

   **Result:** Pass

6. **test-TC6 : Patient Registration (Inactive Invite Code)**

   **Initial State:** App launched; `inviteCodes` contains a matching code with `active=false`.

   **Input:** `registerPatient()` called with an inactive invite code.

   **Output:** Error "Invite code inactive" thrown. No Auth user or Firestore writes created.

   **Result:** Pass

7. **test-TC7 : Patient Registration (Already Used Invite Code)**

   **Initial State:** App launched; `inviteCodes` contains a matching code with `used=true`.

**Input:** `registerPatient()` called with an already used invite code.

**Output:** Error "Invite code already used" thrown. No Auth user or Firestore writes created.

**Result:** Pass

8. **test-TC8 : Patient Registration (Invite Code Missing Physio Link)**

   **Initial State:** App launched; `inviteCodes` contains a matching code with no `physioId` field.

   **Input:** `registerPatient()` called with a code missing a `physioId`.

   **Output:** Error "Invite code missing physio link" thrown. No Auth user or Firestore writes created.

   **Result:** Pass

9. **test-TC9 : User Login (Valid Physiotherapist Credentials)**

   **Initial State:** Login screen available; Firestore contains a profile with `role="physio"`.

   **Input:** `loginUser()` called with valid PT credentials.

   **Output:** Login returns object with `uid` and profile data including `role="physio"`. No errors thrown.

   **Result:** Pass

10. **test-TC10 : User Login (Valid Patient Credentials)**

    **Initial State:** Login screen available; Firestore contains a profile with `role="patient"`.

    **Input:** `loginUser()` called with valid patient credentials.

    **Output:** Login returns object with `uid` and profile data including `role="patient"`. No errors thrown.

    **Result:** Pass

11. **test-TC11 : User Login (Missing Firestore Profile)**

    **Initial State:** Login screen available; valid Firebase Auth credentials exist but no Firestore profile.

**Input:** `loginUser()` called with valid credentials.

**Output:** Error "User profile missing in Firestore" thrown. Login rejected.

**Result:** Pass

12. **test-TC12 : User Logout**

    **Initial State:** User session active.

    **Input:** `logoutUser()` called.

    **Output:** Auth sign-out called once and completes without error.

    **Result:** Pass

13. **test-TC13 : Get User Data**

    **Initial State:** Firestore populated with test user documents.

    **Input:** `getUserData()` called with a valid uid, a non-existent uid, and an empty uid.

    **Output:** Valid uid returns `UserData`. Non-existent uid returns null. Empty uid throws an error.

    **Result:** Pass

14. **test-TC14 : User Lookup by Email**

    **Initial State:** Firestore populated with test user documents.

    **Input:** `getUserByEmail()` called with a registered email and an unregistered email.

    **Output:** Registered email returns user object and true. Unregistered email returns null and false.

    **Result:** Pass

15. **test-TC15 : Update User Data**

    **Initial State:** Firestore populated with test user documents.

    **Input:** `updateUser()` called with a valid update payload.

    **Output:** Update returns true. Changes reflected in Firestore.

    **Result:** Pass

16. **test-TC16 : Delete User**

    **Initial State:** Firestore populated with test user documents.

    **Input:** `deleteUser()` called with a valid uid, a valid email, and a non-existent case.

    **Output:** Valid cases return true. Non-existent case returns false.

    **Result:** Pass

17. **test-TC17 : User Queries**

    **Initial State:** Firestore populated with linked PT and patient records.

    **Input:** `getPatientsByPhysioId()`, `searchByName()`, and `getAllUsers()` called.

    **Output:** Each function returns correctly filtered arrays. Errors return empty arrays.

    **Result:** Pass

18. **test-TC18 : User Info Lookup**

    **Initial State:** Firestore populated with linked PT and patient records.

    **Input:** `getUserdbInfo()` called with name only and with name and birthday filter.

    **Output:** Returns categorized patients and physiotherapists correctly. Birthday filter works as expected.

    **Result:** Pass

19. **test-TC19 : PT Account Delete**

    **Initial State:** Firestore populated with a valid physiotherapist and linked patient.

    **Input:** `deletePTAccount()` called with a valid physio, a non-physio user, and a non-existent patient.

    **Output:** Valid case deletes patient successfully. Non-physio and non-existent cases throw appropriate errors.

    **Result:** Pass

20. **test-TC20 : User Authentication Validation**

    **Initial State:** Firestore populated with test user credentials.

    **Input:** `usernamePwMatch()` and `authenticateUser()` called with valid credentials, an invalid password, and a non-existent user.

    **Output:** Valid credentials return true and user object. Invalid password throws an error. Non-existent user returns null.

    **Result:** Pass

21. **test-TC21 : System Initialization**

    **Initial State:** System not yet initialized.

    **Input:** `initializeSystem()` called.

    **Output:** Success message logged to console. No errors thrown.

    **Result:** Pass

22. **test-TC22 : Credential Validation**

    **Initial State:** User account exists in Firestore.

    **Input:** `validateCredentials()` called with a correct password and an incorrect password.

    **Output:** Correct password returns true. Incorrect password returns false.

    **Result:** Pass

# 5 Nonfunctional Requirements Evaluation

## 5.1 Usability

1. **UT-1 : Navigation Clarity**

   **Initial State:** App installed on a mobile device; user logged in.

   **Input:** User was asked to find and open the exercise screen, start a recording session, and return to the dashboard without assistance.

   **Output:** User successfully navigated through the required screens without confusion. Feedback indicated labels and buttons were clear.

**Result:** Pass

2. **UT-2 : Instruction Clarity During Exercise**

   **Initial State:** User is on the record exercise screen with camera access enabled.

   **Input:** User initiated an exercise session. The application displayed live on-screen feedback instructing the user of any adjustments.

   **Output:** User reported that instructions were clear and prompts matched the required actions. Minor wording improvements were noted and updated.

   **Result:** Pass

3. **UT-3 : Survey Questionnaire – ui_testing**

   **Initial State:** App accessible to testers on mobile devices.

   **Input:** Five users completed a short usability questionnaire addressing ease of use, clarity and navigation.

   **Output:** Average rating across categories was $\geq 4/5$. Minor UI refinements (font size and button spacing) were applied based on feedback.

   **Result:** Pass

4. **test-UT-4 : Account Creation (Usability)**

   **Initial State:** App installed on participant's device; invite code and PT license number provided.

   **Input:** Participant launched the app, selected their role, and completed account creation.

   **Output:** All participants created accounts successfully without assistance. PT license and invite code fields completed without difficulty. Minor font size feedback noted and resolved.

   **Result:** Pass

5. **test-UT-5 : Exercise Recording Initiation**

   **Initial State:** Patient logged in on mobile device; camera permission granted; backend server running.

**Input:** Participant navigated to the Record screen and pressed Start Recording.

**Output:** Camera activated and pose detection began. On-screen prompts instructed participant to enter the detectable frame area. Navigation and button labels reported as clear.

**Result:** Pass

6. **test-UT-6 : Real-Time Pose Detection and Form Feedback**

   **Initial State:** Patient positioned within camera frame; pose detection active; backend returning joint angle and ROM metrics.

   **Input:** Participant performed knee extension repetitions.

   **Output:** App detected relevant body landmarks and displayed appropriate form feedback overlays. Rep count incremented correctly. Detection accuracy was reported as satisfactory.

   **Result:** Pass

7. **test-UT-7 : Exercise Page (Instructions)**

   **Initial State:** Patient logged in; Exercises tab accessible.

   **Input:** Participant navigated to the Exercises page and reviewed listed exercises.

   **Output:** Exercises displayed with names, target muscle groups, set/rep counts, and step-by-step instructions. Instructions reported as clear and easy to follow.

   **Result:** Pass

8. **test-UT-8 : Progress Screen (Session History)**

   **Initial State:** Patient logged in with at least one completed session stored in Firestore.

   **Input:** Participant navigated to the Progress tab.

   **Output:** Session history displayed with ROM and rep count data. Layout was readable and logically arranged. Participants suggested adding sets and pain level as future enhancements.

   **Result:** Pass

9. **test-UT-9 : PT Dashboard (Patient List and Activity)**

   **Initial State:** Physiotherapist logged in; at least one linked patient with recorded sessions.

   **Input:** PT participant reviewed the home dashboard and selected a patient to view their activity history.

   **Output:** Dashboard displayed linked patients and activity history. Patient detail view showed session history and ROM chart. Layout reported as clear and sufficient for clinical use.

   **Result:** Pass

10. **test-UT-10 : PT Feedback Submission**

    **Initial State:** Physiotherapist logged in; patient session detail page accessible.

    **Input:** PT participant selected a session entry and submitted a written comment.

    **Output:** Feedback saved to Firestore and associated with the session. Comment visible from the patient's session view. PT participants rated the feature as clinically useful.

    **Result:** Pass

## 5.2 Performance

1. **PR-1 : Exercise Interface Response Time – ex_int_speed**

   **Initial State:** User logged in on a mobile device; backend server running.

   **Input:** User started recording from the record screen. Screen recording timestamps were used to measure the time from tapping "Start" to the first visible feedback/result. There were five trials conducted.

   **Output:** Measured response times were 1.87s, 1.92s, 1.79s, 1.95s, and 1.84s. The average end-to-end response time was 1.87 seconds ($\leq$ 2 seconds).

   **Result:** Pass

2. **PR-2 : Repeated Use Performance**

   **Initial State:** Application running normally on mobile device.

   **Input:** User completed 10 consecutive "Start → Record Exercise → Stop" cycles.

   **Output:** There was no significant slowdown observed across the 10 trials. Additionally, the response times were all within acceptable range and no crashes took place.

   **Result:** Pass

## 5.3 Reliability and Fault-Tolerance

1. **RFT-1 : Backend Failure Handling – fail_recovery**

   **Initial State:** User actively using the application while connected to backend.

   **Input:** Backend server was manually terminated during active analysis.

   **Output:** Application displayed a network error message and remained responsive (no crash). After backend restart, the user was able to retry and continue normal operation.

   **Result:** Pass

2. **RFT-2 : Network Interruption During Upload – Interrupted_vid**

   **Initial State:** User uploading recorded exercise video.

   **Input:** WiFi connection was disabled mid-upload and restored after approximately 10 seconds.

   **Output:** Application detected upload failure and displayed a retry option. User retried successfully and uploaded video was verified to be intact.

   **Result:** Pass

## 5.4 Security

1. **SR-1 : Unauthorized Access to Protected Endpoint**

**Initial State:** Backend server running with authentication enabled.

**Input:** A request was sent to a protected endpoint without authentication credentials.

**Output:** Server returned HTTP 401 Unauthorized. Additionally, no protected data was returned.

**Result:** Pass

2. **SR-2 : Invalid Credentials Login**

   **Initial State:** Application login screen available.

   **Input:** Login attempted using an email not registered in the database or incorrect password.

   **Output:** System denied login and displayed an appropriate error message without exposing any system details.

   **Result:** Pass

## 5.5 Maintainability and Support

1. **MS-1 : Reproducible Setup**

   **Initial State:** Fresh machine with no prior installation.

   **Input:** Project repository was cloned and installed using instructions provided in README (the frontend and backend setup).

   **Output:** Application ran successfully without missing dependencies. All documented setup steps were sufficient to successfully reproduce the system.

   **Result:** Pass

2. **MS-2 : Code Review and Quality Assurance**

   **Initial State:** Revision 0 completed.

   **Input:** Numerous peer code reviews were conducted through pull requests and issue tracking. The unit tests were executed to validate core functionality, including the metric calculations and functional logic. The UI components were reviewed for usability, clarity of instructions,

and error handling. Moreover, the identified issues were documented and resolved.

**Output:** The core functions passed unit testing. The UI refinements were implemented based on peer feedback. The code structure maintained clear separation between modules and frontend components. Overall, there were no major functional issues identified during review.

**Result:** Pass

# 6 Comparison to Existing Implementation

N/A.

# 7 Unit Testing

The following section outlines the unit tests created for all of the modules.

## 7.1 Authentication

### 7.1.1 Account Creation Tests

The following tests in Table 1 verify the functionality of the account creation module for both physiotherapists and patients, including validation checks and successful user creation workflows.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC1 | FR1.1, FR1.2 | Register physiotherapist with valid license format and a license that exists in `validLicenses` with `active=true`. | Auth user is created. Firestore `users/{uid}` document is written with `role="physio"`, trimmed name, lowercased email, uppercased license number, `verified=true`, and `createdAt`. | User created successfully and required data written to Firestore as expected. All assertions passed and matched expected behaviour. | Pass |
| TC2 | FR1.1, FR1.2 | Register physiotherapist with an invalid license format (e.g., a missing dash or incorrect digits). | Error `"Invalid license format (example: ON-123456)."` is thrown. No Auth user is created. No Firestore writes occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC3 | FR1.1, FR1.2 | Register physiotherapist with valid format but license is not found in `validLicenses` or `active != true`. | Error `"License not verified."` is thrown. No Auth user is created. No Firestore writes occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |
| TC4 | FR1.1, FR1.3, FR2, FR3 | Register patient with invite code that exists in `inviteCodes`, is `active=true`, `used=false`, and includes a `physioId`. | Auth user is created. Firestore `users/{uid}` document is written with `role="patient"`, trimmed name, lowercased email, stored `physioId`, normalized invite code, and `createdAt`. Invite is updated with `used=true` and `usedBy={uid}`. | User created successfully and related records updated as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC5 | FR1.3, FR3 | Register patient with an invite code that does not exist in `inviteCodes`. | Error `"Invalid invite code."` is thrown. No Auth user is created. No Firestore writes/updates occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |
| TC6 | FR1.3, FR3 | Register patient with an invite code that exists but `active=false`. | Error `"Invite code inactive."` is thrown. No Auth user is created. No Firestore writes/updates occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC7 | FR1.3, FR2 | Register patient with an invite code that exists but `used=true`. | Error `"Invite code already used."` is thrown. No Auth user is created. No Firestore writes/updates occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |
| TC8 | FR1.3, FR2 | Register patient with an invite code that exists but missing `physioId`. | Error `"Invite code missing physio link."` is thrown. No Auth user is created. No Firestore writes/updates occur. | Expected error was correctly returned and no unintended operations occurred. All assertions passed and matched expected behaviour. | Pass |

Table 1: Account Creation Test Cases

### 7.1.2 Valid User Tests

The following tests in Table 2 verify the functionality of the valid user module.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC9 | FR1, FR1.1, SR-AR1 | Login with valid credentials for a physiotherapist whose Firestore profile `users/{uid}` exists and contains `role="physio"`. | Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur. | Login returned correct uid and physiotherapist data as expected. All assertions passed and matched expected behaviour. | Pass |
| TC10 | FR1, FR1.1, SR-AR1 | Login with valid credentials for a patient whose Firestore profile `users/{uid}` exists and contains `role="patient"`. | Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur. | Login returned correct uid and patient data as expected. All assertions passed and matched expected behaviour. | Pass |
| TC11 | FR1, FR1.1, SR-AR1 | Login with valid credentials where the Firestore profile `users/{uid}` does not exist. | Error `"User profile missing in Firestore."` is thrown and login is rejected. | Expected error was correctly returned and login was rejected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC12 | | Logout after a user session is active. | The Auth sign-out operation is called once and completes without error. | Sign-out completed successfully without errors. All assertions passed and matched expected behaviour. | Pass |

Table 2: Valid User Test Cases

## 7.2 User Account Tests

The tests below outline the User Account module functionality. Note that many new functions were added for this iteration. Currently, it serves as an interface between the database and the various other components. Additionally, there may be some overlap between the Account Creation module and Valid User module. While the modules themselves perform a few different functions, they are inextricably linked due to their handling of the same entity. For example, the Valid User module consists of tests the ensure successful Login and the User Account module test cases extend that to testing for invalid and non-existent users.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| **USER MANAGEMENT FUNCTIONS** | | | | | |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC13 | FR23, FR21 | Get User Data: Test with valid `uid`, non-existent `uid`, and empty `uid`. | Valid `uid` returns `UserData`; non-existent returns null; empty throws error. | Correct data was returned for valid and invalid inputs as expected. All assertions passed and matched expected behaviour. | Pass |
| TC14 | FR21 | User Lookup: Test get user by email and email existence check with registered and unregistered emails. | Registered email returns user and true; unregistered returns null and false. | Lookup results matched expected values for registered and unregistered emails. All assertions passed and matched expected behaviour. | Pass |
| TC15 | FR2, FR6, FR22 | Update User: Test update user data. | Valid update returns true. | Update operation completed successfully and returned expected result. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC16 | FR4 | Delete User: Test delete by `uid` and by email with valid and non-existent cases. | Valid cases return true; non-existent return false. | Delete operation behaved correctly for valid and invalid cases. All assertions passed and matched expected behaviour. | Pass |
| **QUERY FUNCTIONS** | | | | | |
| TC17 | FR18 | User Queries: Test get patients by physioId, search by name, and get all users. | Returns correctly filtered arrays; errors return empty arrays. | Query results returned correct filtered data and handled errors as expected. All assertions passed and matched expected behaviour. | Pass |
| TC18 | FR21, FR25 | User Info Lookup: Test getUserdbInfo with name only and with name+birthday filter. | Returns categorized patients/physios correctly; filtering works as expected. | Lookup results were correctly categorized and filtered as expected. All assertions passed and matched expected behaviour. | Pass |
| **SPECIALIZED FUNCTIONS** | | | | | |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC19 | FR2, FR4, FR22 | `PT_Account_Delete`: Test with valid physio, non-physio user, and non-existent patient. | Valid case deletes patient; non-physio throws error; non-existent throws error. | Delete operation handled valid and invalid cases correctly with expected outcomes. All assertions passed and matched expected behaviour. | Pass |
| TC20 | SR-AR1, SR-PR1 | Test userNamePwMatch and authenticateUser with valid credentials, invalid password, and non-existent user. | Valid returns true/user; invalid throws appropriate errors; non-existent returns null. | Authentication results matched expected outcomes for all cases. All assertions passed and matched expected behaviour. | Pass |
| TC21 | SR-AR1, SR-AR2 | System Initialization: Test initializeSystem. | Logs success message to console. | System initialized successfully and logged expected output. All assertions passed and matched expected behaviour. | Pass |
| **VALIDATION HELPERS** | | | | | |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC22 | FR1, SR-AR1 | Test validate-Credentials with correct and incorrect passwords. | Correct returns true; incorrect returns false. | Validation results correctly distinguished valid and invalid credentials. All assertions passed and matched expected behaviour. | Pass |

Table 3: User Account Service Test Cases

## 7.3 Live Feedback Tests

The following tests in Table 4 verify the functionality of the live feedback module, including real-time corrective message generation, safety warning triggers, rep counting, and physiotherapist comment submission and retrieval.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC23 | FR13, FR14 | Submit joint angle measurements within the acceptable range for a knee extension rep. | No corrective feedback message is generated. Rep count increments by one. Positive reinforcement message returned. | Feedback and rep count behaved as expected for valid input. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC24 | FR13, FR14 | Submit joint angle measurements below the minimum threshold for knee extension (insufficient range of motion). | Corrective feedback message is generated (e.g., `"Extend your knee further to complete the rep."`). Rep is not counted. | Corrective feedback was returned and rep was not counted as expected. All assertions passed and matched expected behaviour. | Pass |
| TC25 | FR13, FR14 | Submit joint angle measurements above the maximum safe threshold, simulating an overextension. | Corrective feedback message is generated (e.g., `"Reduce the range of your movement."`). Rep is not counted. | Corrective feedback was returned and rep was not counted as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC26 | FR16, PR-SCR1 | Submit joint angle measurements that exceed the defined dangerous movement threshold. | Safety warning is triggered (e.g., `"Stop immediately \| dangerous movement detected."`). Exercise session is halted. | Safety warning was triggered and session was halted as expected. All assertions passed and matched expected behaviour. | Pass |
| TC27 | FR14, PR-SL2 | Submit a sequence of 5 consecutive valid reps and verify feedback generation latency for each. | Feedback is returned for each rep within 2 seconds. Rep count reflects 5 completed reps. No degradation in response time across reps. | Feedback latency and rep count met expected performance requirements. All assertions passed and matched expected behaviour. | Pass |
| TC28 | FR14 | Invoke the feedback generator with no pose landmarks detected (user out of frame). | Feedback message returned is `"Please step into the camera frame."` No rep is counted. No safety warning triggered. | Correct feedback was returned and no rep was counted as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC29 | FR20, FR25 | Physiotherapist submits a written comment on a completed patient session. | Comment is saved to Firestore and associated with the correct session document. No errors thrown. | Comment was saved and linked to the correct session as expected. All assertions passed and matched expected behaviour. | Pass |
| TC30 | FR19, FR25 | Patient retrieves physiotherapist feedback for a session that has an associated PT comment. | Correct comment text is returned and associated with the matching session. Non-existent session returns null. | Correct comment was retrieved and invalid cases handled as expected. All assertions passed and matched expected behaviour. | Pass |

Table 4: Live Feedback Test Cases

## 7.4 Video Capture and Review Tests

The tests below outline the Video Capture and Review functionality. Some of the interactions with following tests as well as other modules may touch on the video capture and review module, but the tests below are focused on the core functionalities of the module itself.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC31 | FR12 | Call `get_pose_connections(legs_only=False)` and verify against `POSE_CONNECTIONS`. | `POSE_CONNECTIONS` and `LEG_CONNECTIONS` have length greater than 0. `LEG_CONNECTIONS` length is less than `POSE_CONNECTIONS`, and the set of connections from `get_pose_connections(False)` equals the set of `POSE_CONNECTIONS`. | Connections returned matched expected full pose connections. All assertions passed and matched expected behaviour. | Pass |
| TC32 | FR12 | Call `get_pose_connections(legs_only=True)` and verify against `LEG_CONNECTIONS`. | The connections returned equal the set of `LEG_CONNECTIONS`. | Connections returned matched expected leg-only connections. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC33 | FR12 | Call `extract_landmarks()` with a valid pose result containing 33 landmarks. | Returns a list of 33 landmarks. Each landmark has x, y, z values that are close to the expected i/33 values. | Landmarks were extracted correctly with expected values. All assertions passed and matched expected behaviour. | Pass |
| TC34 | FR26 | Call `extract_landmarks()` with None and with an empty result `pose_landmarks= None`. | Returns None for both. | Function correctly returned None for invalid inputs. All assertions passed and matched expected behaviour. | Pass |
| TC35 | FR7 | Initialize PoseCamera with mocked VideoCapture and mocked PoseLandmarker. | Camera initializes successfully. `display_size` is set to (640, 480). `cap.set` is called to configure frame dimensions. | Camera initialized and configured as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC36 | FR7 | Call `get_frame()` with mocked camera that returns a numpy array frame. | Returns (True, frame) where frame is a numpy array. Frame is resized to display dimensions. | Frame was retrieved and resized correctly. All assertions passed and matched expected behaviour. | Pass |
| TC37 | FR12 | Call `process_pose()` with a mocked frame. | Returns a mock result. The pose detector's `detect` method is called once with the converted RGB image. | Pose processing returned expected result and called detector correctly. All assertions passed and matched expected behaviour. | Pass |
| TC38 | FR14 | Call `draw_landmarks()` with a mock result and a numpy array frame. | Returns the frame. OpenCV's `line` and `circle` functions are called to draw the pose landmarks. | Landmarks were drawn correctly on frame. All assertions passed and matched expected behaviour. | Pass |

Table 5: Video Capture and Review Test Cases

## 7.5 Computer Vision and Form Analysis Tests

The following tests in Table 6 verify the functionality of the computer vision and form analysis module by ensuring that joint angles are accurately calculated, knee angles are correctly extracted from pose landmarks, the analyzer calibrates to the user's range of motion, repetitions are counted properly, and summary metrics are generated as expected.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC39 | FR12 | Call `angle_deg()` with points forming a 90-degree angle: a=(0,1), b=(1,1), c=(1,0). | Returns 90.0 degrees (within 0.01 tolerance). | Angle calculation returned expected value within tolerance. All assertions passed and matched expected behaviour. | Pass |
| TC40 | FR12 | Call `angle_deg()` with points forming a 45-degree angle: a=(0,1), b=(1,1), c=(0,0). | Returns 45.0 degrees (within 0.01 tolerance). | Angle calculation returned expected value within tolerance. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC41 | FR12 | Call `angle_deg()` with points forming a 180-degree straight line: a=(0,1), b=(1,1), c=(2,1). | Returns 180.0 degrees (within 0.01 tolerance). | Angle calculation returned expected value within tolerance. All assertions passed and matched expected behaviour. | Pass |
| TC42 | FR26 | Call `angle_deg()` with None values. | Returns None for all three cases. | Function correctly handled invalid inputs and returned None. All assertions passed and matched expected behaviour. | Pass |
| TC43 | FR12 | Call `knee_angle_from_result()` (right leg). | Returns valid angle. | Knee angle was computed correctly from valid input. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC44 | FR12 | Call `knee_angle_from_result()` (left leg). | Returns valid angle. | Knee angle was computed correctly from valid input. All assertions passed and matched expected behaviour. | Pass |
| TC45 | FR26 | Call `knee_angle_from_result()` with None. | Returns None. | Function correctly returned None for invalid input. All assertions passed and matched expected behaviour. | Pass |
| TC46 | FR26 | Call with `pose_landmarks=None`. | Returns None. | Function correctly handled missing landmarks. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC47 | FR26 | Missing right knee landmark. | Returns None. | Missing landmark was handled correctly. All assertions passed and matched expected behaviour. | Pass |
| TC48 | FR26 | Missing left knee landmark. | Returns None. | Missing landmark was handled correctly. All assertions passed and matched expected behaviour. | Pass |
| TC49 | FR26 | Empty landmarks. | Returns None. | Empty input was handled correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC50 | FR12 | Initialize Analyzer. | Default values set. | Analyzer initialized with correct default state. All assertions passed and matched expected behaviour. | Pass |
| TC51 | FR8, FR27 | Start calibration. | Calibration fields updated. | Calibration started and parameters set correctly. All assertions passed and matched expected behaviour. | Pass |
| TC52 | FR8, FR27 | Calibration updates. | Min/max updated. | Calibration values updated correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC53 | FR8, FR27 | Calibration timing. | Calibration completes. | Calibration timing handled correctly and state updated. All assertions passed and matched expected behaviour. | Pass |
| TC54 | FR8, FR26 | Calibration with None. | Ignored. | Invalid input ignored during calibration as expected. All assertions passed and matched expected behaviour. | Pass |
| TC55 | FR8, FR16 | Set analyzer state. | Valid state. | Analyzer state initialized correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC56 | FR8, FR14 | Extension → Flexion. | State change. | Rep state transitioned correctly. All assertions passed and matched expected behaviour. | Pass |
| TC57 | FR8, FR11 | Flexion → Extension. | Rep counted. | Rep counted and duration recorded correctly. All assertions passed and matched expected behaviour. | Pass |
| TC58 | FR8, FR11 | Missing current rep. | No count. | Rep was not counted as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC59 | FR8, FR12 | Min/max expansion. | Updated values. | Min and max values updated correctly. All assertions passed and matched expected behaviour. | Pass |
| TC60 | FR8, FR27 | Calibration update call. | Function called. | Calibration update was triggered correctly. All assertions passed and matched expected behaviour. | Pass |
| TC61 | FR8, FR12 | Initial update. | Set min/max. | Initial values set correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC62 | FR8, FR27 | Update lower bound. | Min updated. | Minimum angle updated correctly. All assertions passed and matched expected behaviour. | Pass |
| TC63 | FR8, FR27 | Update upper bound. | Max updated. | Maximum angle updated correctly. All assertions passed and matched expected behaviour. | Pass |
| TC64 | FR8, FR26 | Update with None. | No change. | State remained unchanged as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC65 | FR11, FR17 | Summary output. | All fields present. | Summary returned expected structure and values. All assertions passed and matched expected behaviour. | Pass |
| TC66 | FR11, FR17 | Summary rounding. | Rounded values. | Summary values were computed and rounded correctly. All assertions passed and matched expected behaviour. | Pass |
| TC67 | FR8, FR27 | Calibration summary. | Time left. | Calibration state and timing reported correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC68 | FR11, FR17 | Non-calibrating summary. | No time left. | Summary correctly reflected non-calibrating state. All assertions passed and matched expected behaviour. | Pass |
| TC69 | FR11, FR17 | No reps. | Zero durations. | Summary handled empty rep data correctly. All assertions passed and matched expected behaviour. | Pass |
| TC70 | FR11, FR8 | Active rep timing. | Correct durations. | Rep duration values calculated correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC71 | FR8, FR12 | Zero ROM. | No errors. | Edge case handled without errors. All assertions passed and matched expected behaviour. | Pass |
| TC72 | FR12, FR26 | Negative or Positive update. | Range expands. | Angle range updated correctly for edge values. All assertions passed and matched expected behaviour. | Pass |
| TC73 | FR12, FR26 | Large angle. | Max expands. | Upper bound updated correctly. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC74 | FR8 | No state change case. | No update. | State remained unchanged as expected. All assertions passed and matched expected behaviour. | Pass |

Table 6: Computer Vision and Form Analysis Test Cases

## 7.6 Performance and Latency Tests

The following tests in Table 7 verify the functionality of the performance and latency module by ensuring that the backend services initialize correctly, API endpoints respond appropriately to valid and invalid requests, frame processing handles mirroring and landmark detection correctly, frontend services communicate with the backend, and exercise metrics are saved to Firestore with proper error handling.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC75 | FR12 | Start the PoseBackend with mocked values. | PoseBackend init values work as intended successfully and `is_calibrating` is set to True. | Backend initialized with `is_calibrating=True` as expected. All assertions passed and matched expected behaviour. | Pass |
| TC76 | FR8 | Call `reset()` method on PoseBackend. | Previous analyzer is replaced with a newer analyzer object. New one has `is_calibrating` set to True. | Previous analyzer was replaced with a new analyzer instance and `is_calibrating=True` was correctly set. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC77 | FR8 | Send POST request to `/reset` endpoint. | Response status 200 with JSON `\{"ok":true\}`. Backend reset method is called. | Response returned status 200 with correct JSON and backend reset method was successfully invoked. All assertions passed and matched expected behaviour. | Pass |
| TC78 | FR7, FR26 | Send POST request to `/process_frame` with empty image data. | Response status 400 with error message `"Missing imageBase64"`. Landmarks is null. | Response returned status 400 with the expected error message and null landmarks for missing input. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC79 | FR7, FR26 | Send POST request to `/process_frame` with invalid base64 image string. | Response status 400 with error message. No frame processing occurs. | Response returned status 400 with error message and no frame processing occurred for invalid input. All assertions passed and matched expected behaviour. | Pass |
| TC80 | FR7, FR12 | Send POST request to `/process_frame` with valid base64 image, `side="RIGHT"`, `mirrored=true`, `legsOnly=true`. Landmarks detected. | Response status 200. Contains all landmarks (33). Metrics contain correct values `angle=45.6`, `rep\_count=5`, `rep\_state="Extension"`. | Response returned status 200 with all 33 landmarks and correct metrics including angle, rep count, and rep state. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC81 | FR7, FR26 | Send POST request to `/process_frame` with valid image but no pose landmarks detected. | Response status 200. Landmarks is null. Default metrics returned with `calibrating=True`. | Response returned status 200 with null landmarks and default metrics including `calibrating=True`. All assertions passed and matched expected behaviour. | Pass |
| TC82 | FR7 | Send POST request to `/process_frame` with `mirrored=True` and verify frame flipping. | Frame is flipped horizontally before processing. `cv2.flip` is called once with the frame. | Frame was correctly flipped horizontally and `cv2.flip` was called once as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC83 | FR7 | Send POST request to `/process_frame` with `mirrored=False` and verify no frame flipping. | Frame is not flipped. `cv2.flip` is not called. | Frame was not flipped and `cv2.flip` was not called as expected. All assertions passed and matched expected behaviour. | Pass |
| TC84 | FR7 | Test `b64_to_bgr_image` helper function with a valid test image. | Function correctly decodes base64 string back to a numpy array with expected dimensions (100, 100, 3). | Function correctly decoded the base64 string into a numpy array with expected dimensions (100, 100, 3). All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC85 | FR12 | Send POST requests with `side="RIGHT"` and `side="LEFT"` parameters. | `knee_angle_from_result` is called twice with correct side parameters: first with 'RIGHT', then with 'LEFT'. | Function was called twice with correct side parameters 'RIGHT' and 'LEFT' as expected. All assertions passed and matched expected behaviour. | Pass |
| TC86 | FR7, FR26 | Send POST request with valid image but simulate decoding error by returning None. | Response status 400 with error message in JSON. | Response returned status 400 with expected error message when decoding failed. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC87 | FR7 | Call `processFrame()` with front camera facing, RIGHT side, and a valid base64 image. Mock successful API response. | Function calls axios.post with correct URL, payload includes `mirrored:true`, `side:"RIGHT"`, `legsOnly:true`. Returns the mocked response data with landmarks, connections, and metrics. | Function called axios.post with correct payload including mirrored, side, and legsOnly values, and returned the expected mocked response data. All assertions passed and matched expected behaviour. | Pass |
| TC88 | FR7 | Call `processFrame()` with back camera facing, LEFT side. | Function calls axios.post with `mirrored: false` in the payload. | Function correctly sent payload with `mirrored: false` for back camera. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC89 | FR7 | Call `processFrame()` and simulate a network error. | Function throws an error with message "Network error". | Function correctly threw a "Network error" when request failed. All assertions passed and matched expected behaviour. | Pass |
| TC90 | FR8 | Call `resetBackend()` and mock successful response. | Function calls axios.post to `/reset` endpoint with timeout 8000ms. Returns successfully. | Function successfully called axios.post to `/reset` with correct timeout and returned as expected. All assertions passed and matched expected behaviour. | Pass |
| TC91 | FR8 | Call `resetBackend()` and simulate a failure. | Function throws an error with message "Reset failed". | Function correctly threw "Reset failed" error on failure. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC92 | FR1, FR1.1 | Call `saveMetrics()` when no user is logged in (getAuth returns null). | Function throws error "User must be logged in to save metrics". | Function correctly threw "User must be logged in to save metrics" when no user was authenticated. All assertions passed and matched expected behaviour. | Pass |
| TC93 | FR1.1, FR21 | Call `saveMetrics()` when user is logged in but user document does not exist in Firestore. | Function throws error "User data not found in Firestore". | Function correctly threw "User data not found in Firestore" when user document was missing. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC94 | FR21, FR11 | Call `saveMetrics()` with valid metrics data and mocked Firebase responses. | Returns document ID "mock-doc-id". Verifies Firestore calls: doc, getDoc, collection, addDoc. Saved data contains all of the correct fields including actid with timestamp. | Function returned "mock-doc-id" and correctly saved all expected fields including metrics, user info, and Firestore interactions. All assertions passed and matched expected behaviour. | Pass |
| TC95 | FR11, FR21 | Call `saveMetrics()` with minimal metrics data (missing optional `avg_rep_duration` and `current_rep_duration`). | Saved data has `avg_rep_duration=0`, `current_rep_duration=0`, `duration=0`. | Function correctly saved default values for missing optional fields including avg_rep_duration and current_rep_duration All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC96 | FR1.1, FR21 | Call `saveMetrics()` when user document exists but has no name field. | Saved data uses "Unknown User" for the name field. | Function correctly used "Unknown User" as fallback for missing name field. All assertions passed and matched expected behaviour. | Pass |
| TC97 | FR1 | Call `saveSession Data()` when there is no user logged in. | Function throws error "User must be logged in". | Function correctly threw "User must be logged in" when no user was authenticated. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC98 | FR21, FR17, FR11 | Call `saveSessionData()` with valid session data. | Returns document ID "mock-doc-id". Saved data contains actid with `"session_"prefix`, `analysis="completed"`, `completed_reps=8`, `duration=1`, `exercise="KneeExtensionSession"`, `max_height=70.5`, `min_height=30.1`, `average_rom=40.4`, `total_reps=8`, `avg_rep_duration=1.2`. | Function returned "mock-doc-id" and correctly saved session data with all expected fields. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC99 | FR11, FR21 | Call `saveSessionData()` without avgRepDuration (uses session duration instead). | Saved data has duration calculated from startTime and endTime (0 seconds), and avg_rep_duration is undefined. | Function correctly calculated duration and handled missing avgRepDuration as expected. All assertions passed and matched expected behaviour. | Pass |
| TC100 | FR1.1, FR21 | Call `saveSessionData()` when user document does not exist. | Saved data uses "Unknown User" for the name field. | Function correctly used "Unknown User" fallback when user document was missing. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC101 | FR1.1, FR21 | Call `getUserDisplayName()` with logged in user and existing user document with name field. | Returns "Test User". Verifies doc and getDoc calls with correct parameters. | Function correctly returned "Test User" and performed expected Firestore calls. All assertions passed and matched expected behaviour. | Pass |
| TC102 | FR1.1, FR21 | Call `getUserDisplayName()` when user document exists but has no name field. | Returns user email `"test@example.com"`. | Function correctly returned user email `"test@example.com"` when name field was missing. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*(Continued from previous page)*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC103 | FR1 | Call `getUser DisplayName()` when no user is logged in. | Returns "Unknown User". | Function correctly returned "Unknown User" when no user was logged in. All assertions passed and matched expected behaviour. | Pass |
| TC104 | FR1.1 | Call `getUser DisplayName()` and simulate a Firestore error. | Returns "Unknown User" (error caught and handled gracefully). | Function correctly handled Firestore error and returned "Unknown User" as fallback. All assertions passed and matched expected behaviour. | Pass |

Table 7: Performance and Latency Test Cases

## 7.7 Profile Activity Tests

The following tests in Table 8 verify the functionality of the profile activity module. This module is responsible for storing, retrieving, and handling

user activity data, stored user data, summary data, and chart data. The tests ensure this functionality works as expected under various conditions, including valid inputs, edge cases, and error scenarios.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC105 | FR21, FR23 | Test getCurrentUser with correct inputs. | Returns the correct user object. | Function returned the correct user object as expected. All assertions passed and matched expected behaviour. | Pass |
| TC106 | FR21, FR23 | Test getCurrentUser with no current user. | Returns null. | Function correctly returned null when no current user was present. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC107 | FR21, FR23 | Test getCurrentUser with non-existent user. | Returns null. | Function correctly returned null for non-existent user. All assertions passed and matched expected behaviour. | Pass |
| TC108 | FR21, FR23 | Test getCurrentUser with Firestore error. | Returns null. | Function correctly handled Firestore error and returned null. All assertions passed and matched expected behaviour. | Pass |
| TC109 | FR21, FR23 | Test getCurrentUserID with correct inputs. | Returns the correct user ID. | Function returned the correct user ID as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC110 | FR21, FR23 | Test getCurrentUserID with no current user. | Returns null. | Function correctly returned null when no user was logged in. All assertions passed and matched expected behaviour. | Pass |
| TC111 | FR10, FR11, FR17, FR18, FR21 | Test getUserActivities with correct inputs. | Returns the correct array of activities. | Function returned the correct array of activities as expected. All assertions passed and matched expected behaviour. | Pass |
| TC112 | FR10, FR11, FR17, FR18, FR21 | Test getUserActivities with no activities. | Returns an empty array. | Function correctly returned an empty array when no activities were found. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC113 | FR10, FR11, FR17, FR18, FR21 | Test getUserActivities with Firestore error. | Returns an empty array. | Function correctly handled Firestore error and returned an empty array. All assertions passed and matched expected behaviour. | Pass |
| TC114 | FR10, FR11, FR17, FR18, FR21 | Test getUserActivities with incorrect uid. | Returns an empty array. | Function correctly returned an empty array for invalid uid. All assertions passed and matched expected behaviour. | Pass |
| TC115 | FR10, FR11, FR17 | Test getUserSummary with correct inputs. | Returns the correct summary data. | Function returned the correct summary data as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC116 | FR10, FR11, FR17 | Test getUserSummary with correct inputs but duplicate dates. | Returns the correct summary data. | Function correctly handled duplicate dates and returned accurate summary data. All assertions passed and matched expected behaviour. | Pass |
| TC117 | FR10, FR11, FR17 | Test getUserSummary with Firestore error. | Returns zero summary data. | Function correctly handled Firestore error and returned zero summary data. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC118 | FR10, FR18 | Test repsChartData with correct inputs. | Returns the correct chart data. | Function returned the correct reps chart data as expected. All assertions passed and matched expected behaviour. | Pass |
| TC119 | FR10, FR18 | Test repsChartData with correct inputs and duplicate dates. | Returns the correct chart data. | Function correctly handled duplicate dates and returned accurate chart data. All assertions passed and matched expected behaviour. | Pass |
| TC120 | FR10, FR18 | Test repsChartData with activities but no reps. | Returns all 0 chart values. | Function correctly returned all zero chart values when no reps were present. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC121 | FR10, FR18 | Test repsChartData with no activites. | Returns all 0 chart values. | Function correctly returned all zero chart values when no activities existed. All assertions passed and matched expected behaviour. | Pass |
| TC122 | FR10, FR18 | Test repsChartData with firestore error. | Returns all 0 chart values. | Function correctly handled Firestore error and returned zero chart values. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC123 | FR10, FR18 | Test exerciseChartData with correct data. | Returns the correct chart data. | Function returned the correct exercise chart data as expected. All assertions passed and matched expected behaviour. | Pass |
| TC124 | FR10, FR18 | Test exerciseChartData with correct data and duplicate dates. | Returns the correct chart data. | Function correctly handled duplicate dates and returned accurate chart data. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC125 | FR10, FR18 | Test exerciseChartData with no activities. | Returns all 0 chart values. | Function correctly returned zero chart values when no activities were present. All assertions passed and matched expected behaviour. | Pass |
| TC126 | FR10, FR18 | Test exerciseChartData with firestore error. | Returns all 0 chart values. | Function correctly handled Firestore error and returned zero chart values. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC127 | FR10, FR11, FR18, FR19, FR22 | Test getComments with correct inputs and data. | Returns the correct comment data. | Function returned the correct comment data as expected. All assertions passed and matched expected behaviour. | Pass |
| TC128 | FR10, FR11, FR18, FR19, FR22 | Test getComments with no existing comments. | Returns an empty array. | Function correctly returned an empty array when no comments existed. All assertions passed and matched expected behaviour. | Pass |
| TC129 | FR10, FR11, FR18, FR19, FR22 | Test getComments with firestore error. | Returns an empty array. | Function correctly handled Firestore error and returned an empty array. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC130 | FR10, FR11, FR18, FR19, FR22 | Test postComment with correct inputs and data. | Calls collection and update functions with expected inputs. | Function correctly called collection and update functions with expected inputs. All assertions passed and matched expected behaviour. | Pass |
| TC131 | FR10, FR11, FR18, FR19, FR22 | Test postComment with an empty comment. | Should not call collection, addDoc, or setDoc. | Function correctly prevented database calls for empty comment input. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC132 | FR10, FR11, FR18, FR19, FR22 | Test postComment with firestore error. | Should not call setDoc, but should proceed until addDoc errors. | Function correctly handled Firestore error and stopped at addDoc failure as expected. All assertions passed and matched expected behaviour. | Pass |
| TC133 | FR18, FR21 | Test getName with correct inputs and data. | Returns the corresponding user's name. | Function returned the correct user's name as expected. All assertions passed and matched expected behaviour. | Pass |
| TC134 | FR18, FR21 | Test getName with an unknown user. | Returns 'Unknown User'. | Function correctly returned "Unknown User" for unknown user. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC135 | FR18, FR21 | Test getName with a user with no name. | Returns 'Unnamed User'. | Function correctly returned "Unnamed User" when name field was missing. All assertions passed and matched expected behaviour. | Pass |
| TC136 | FR18, FR21 | Test getName with firestore error. | Returns 'Unknown User'. | Function correctly handled Firestore error and returned "Unknown User". All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC137 | FR17, FR18, FR19, FR22 | Test setSelectedUserID with a valid userID. | Calls AsyncStorage with expected inputs. | Function correctly called AsyncStorage with expected inputs. All assertions passed and matched expected behaviour. | Pass |
| TC138 | FR17, FR18, FR19, FR22 | Test setSelectedUserID with an empty userID. | Does not call AsyncStorage. | Function correctly avoided calling AsyncStorage for empty userID. All assertions passed and matched expected behaviour. | Pass |
| TC139 | FR17, FR18, FR19, FR22 | Test getSelectedUserID when a userID is stored. | Returns the correct stored userID. | Function correctly returned the stored userID. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC140 | FR17, FR18, FR19, FR22 | Test getSelectedUserID when no userID is stored. | Returns null. | Function correctly returned null when no userID was stored. All assertions passed and matched expected behaviour. | Pass |
| TC141 | FR17, FR18, FR19, FR22 | Test getSelectedUser when a user is stored. | Returns the stored UserData structure. | Function correctly returned the stored UserData structure. All assertions passed and matched expected behaviour. | Pass |
| TC142 | FR17, FR18, FR19, FR22 | Test getSelectedUser when no user is stored. | Returns null. | Function correctly returned null when no user was stored. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC143 | FR17, FR18, FR19, FR22 | Test clearSelectedUserID on AsyncStorage. | Calls AsyncStorage.removeItem with expected inputs. | Function correctly called AsyncStorage.removeItem with expected inputs. All assertions passed and matched expected behaviour. | Pass |
| TC144 | FR2, FR3 | Test getPhysioInviteCode when an invite code exists. | Returns the corresponding physio invite code. | Function correctly returned the corresponding physio invite code. All assertions passed and matched expected behaviour. | Pass |
| TC145 | FR2, FR3 | Test getPhysioInviteCode when no invite code exists. | Returns null. | Function correctly returned null when no invite code existed. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC146 | FR2, FR3 | Test getPhysioInviteCode when userID is invalid. | Returns null. | Function correctly returned null for invalid userID. All assertions passed and matched expected behaviour. | Pass |

Table 8: Profile Activity Test Cases

## 7.8 Exercise Data Tests

The following tests in Table 9 verify the functionality of the exercise data module. This module is responsible for storing, retrieving, and handling user exercise data, general exercise data for assignable exercises by physios, and assigned exercises for given users. This function also facilitates the modification, addition, and deletion of exercises for users by their assigned physios. The tests outlined in the table ensure that all of this functionality works as expected under various conditions, including valid inputs, edge cases, and error scenarios.

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC147 | FR5, FR18, FR21, FR27 | Tests getExercisesById with valid userID and exercise data | Returns the corresponding user's exercises. | Function returned the corresponding user's exercises as expected. All assertions passed and matched expected behaviour. | Pass |
| TC148 | FR5, FR18, FR21, FR27 | Tests getExercisesById with valid userID but no exercise data | Returns undefined. | Function correctly returned undefined when no exercise data existed. All assertions passed and matched expected behaviour. | Pass |
| TC149 | FR5, FR18, FR21, FR27 | Tests getExercisesById with firestore error | Returns undefined. | Function correctly handled Firestore error and returned undefined. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC150 | FR5, FR18, FR21, FR27 | Tests getExercise with valid exerciseID | Returns the corresponding exercise. | Function returned the corresponding exercise as expected. All assertions passed and matched expected behaviour. | Pass |
| TC151 | FR5, FR18, FR21, FR27 | Tests getExercise with an invalid exerciseID | Returns undefined. | Function correctly returned undefined for invalid exerciseID. All assertions passed and matched expected behaviour. | Pass |
| TC152 | FR5, FR18, FR21, FR27 | Tests getExercise with firestore error | Returns undefined. | Function correctly handled Firestore error and returned undefined. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC153 | FR5, FR18, FR21, FR27 | Tests getGeneralExercises when general exercises exist | Returns a list of all exercises. | Function returned a list of all exercises as expected. All assertions passed and matched expected behaviour. | Pass |
| TC154 | FR5, FR18, FR21, FR27 | Tests getGeneralExercises when no general exercises exist | Returns undefined. | Function correctly returned undefined when no general exercises existed. All assertions passed and matched expected behaviour. | Pass |
| TC155 | FR5, FR18, FR21, FR27 | Tests getGeneralExercises with firestore error | Returns undefined. | Function correctly handled Firestore error and returned undefined. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC156 | FR6, FR18, FR21 | Tests removeUserExercise with valid userID and exercise data | Calls deleteDoc with the expected inputs. | Function correctly called deleteDoc with expected inputs. All assertions passed and matched expected behaviour. | Pass |
| TC157 | FR6, FR18, FR21 | Tests removeUserExercise with valid userID but invalid exercise data | Does not call deleteDoc. | Function correctly did not call deleteDoc for invalid exercise data. All assertions passed and matched expected behaviour. | Pass |
| TC158 | FR6, FR18, FR21 | Tests removeUserExercise with firestore error | Does not call deleteDoc. | Function correctly handled Firestore error and did not call deleteDoc. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC159 | FR6, FR18, FR21 | Tests addUserExercise with valid userID and exercise data | Calls addDoc and updateDoc with the expected inputs. | Function correctly called addDoc and updateDoc with expected inputs. All assertions passed and matched expected behaviour. | Pass |
| TC160 | FR6, FR18, FR21 | Tests addUserExercise with firestore error | Does not call updateDoc. | Function correctly handled Firestore error and did not call updateDoc. All assertions passed and matched expected behaviour. | Pass |
| TC161 | FR6, FR18, FR21 | Tests updateUserExercise with valid userID and exercise data | Calls updateDoc with expected inputs. | Function correctly called updateDoc with expected inputs. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC162 | FR6, FR18, FR21 | Tests updateUserExercise with valid userID but invalid exercise data | Does not call updateDoc with bad inputs. | Function correctly did not call updateDoc for invalid exercise data. All assertions passed and matched expected behaviour. | Pass |
| TC163 | FR6, FR18, FR21 | Tests updateUserExercise with firestore error | Does not call updateDoc. | Function correctly handled Firestore error and did not call updateDoc. All assertions passed and matched expected behaviour. | Pass |
| TC164 | FR5, FR18, FR21, FR27 | Tests getSelectedExercises with valid user ID and exercise data. | Returns corresponding exercise data. | Function returned corresponding exercise data as expected. All assertions passed and matched expected behaviour. | Pass |

*Continued on next page*

*Continued from previous page*

| ID | Ref. Req. | Action | Expected Result | Actual Result | Result |
|---|---|---|---|---|---|
| TC165 | FR5, FR18, FR21, FR27 | Tests getSelectedExercises with valid user ID but no exercise data. | Returns undefined. | Function correctly returned undefined when no exercise data existed. All assertions passed and matched expected behaviour. | Pass |
| TC166 | FR5, FR18, FR21, FR27 | Tests getSelectedExercises with firestore error | Returns undefined. | Function correctly handled Firestore error and returned undefined. All assertions passed and matched expected behaviour. | Pass |

Table 9: Exercise Data Test Cases

# 8 Changes Due to Testing

## 8.1 Usability Testing Results

Analyses from usability testing revealed important user experience insights which has provided clarity with what should be the focus for further development. A subset of recommendations will be developed further, while others will be tabled as out of scope due to project timeline constraints. There were

a total of 5 participants in the study, with 2 being PTs, and the rest were those who satisfied the pool criteria. The diverse perspectives proved to be essential by honing in on elements that may provide substantial added value to the application. Both user flows (PT and patient) were tested with the corresponding participants.
The physiotherapist participants had 2 recommendations: A more comprehensive exercise activity record (eg. adding sets in addition to reps) for a clearer picture on the patient's performance, and inclusion of a post-exercise feedback form to provide an insight into performance. (eg. modifying the rep count due to patient's increased pain).
The other participants highlighted the need for audio cues while performing the exercise. Reading the instructions from the acceptable distance necessary for proper application function was troublesome. Secondly, additional data on the Progress tab would be beneficial for patients to keep track, in a non-technical context, of their recovery.
Please refer to the Usability Report for additional information.

# 9 Automated Testing

In terms of automated testing, all of the tests for the TypeScript frontend modules as well as the Python backend are automated. When it comes to running the tests, the Python tests are run using `Pytest` by simply typing `pytest` in the command line in the correct root project directory. These tests validate core backend functionality such as the metric calculations and helper functions utilized during exercise processing. Additionally, all of the TypeScript tests can be run in the same manner, however, they run with `Jest` and through running the following command: `npm run test` instead. Ultimately, these tests focus on validating frontend logic including authentication and user account management, user activity tracking, exercise data handling, performance statistics generation, and the feedback module that displays live feedback to users and allows both the physiotherapistss and patients the opportunity to leave comments regarding the user's respective session. It is important to note that the both the Firebase calls and camera components are mocked during testing so that we are able to separately test the logic functionality. Also, the output results for both the frontend and backend automated test suites are printed to the console, ultimately al-

lowing us to be able to quickly identify both the passing and failing tests immediately.

# 10 Trace to Requirements

The following section showcases the mapping between the various tests performed to the specific requirements that they validate, ultimately displaying the traceability to requirements.

Table 10: Functional Requirements and Corresponding Test Cases

| **Test Cases** | **Functional Requirement(s)** |
|---|---|
| *Account Creation Tests* | |
| TC1-3 | FR1.1, FR1.2 |
| TC4-8 | FR1.1, FR1.3, FR2, FR3 |
| TC9-11 | FR1, FR1.1 |
| *User Account Tests* | |
| TC13-16 | FR2, FR4, FR6, FR21, FR22, FR23 |
| TC17-18 | FR18, FR21, FR25 |
| TC19-21 | FR2, FR4, FR22, SR-AR1, SR-AR2, SR-PR1 |
| TC22 | FR1, SR-AR1 |
| *Live Feedback Tests* | |
| TC23-25 | FR13, FR14 |
| TC26 | FR16 |
| TC27-28 | FR14 |
| TC29 | FR20, FR25 |
| TC30 | FR19, FR25 |
| *Video Capture and Review Tests* | |

| | |
|---|---|
| TC31-33, 37 | FR12 |
| TC34 | FR26 |
| TC35-36 | FR7 |
| TC38 | FR14 |
| *Computer Vision and Form Analysis Tests* | |
| TC39-41, 43, 44, 50 | FR12 |
| TC42, 45-49 | FR26 |
| TC51-53, 60, 62, 63, 67 | FR8, FR27 |
| TC54, 64 | FR8, FR26 |
| TC55 | FR8, FR16 |
| TC56 | FR8, FR14 |
| TC57, 58, 70 | FR8, FR11 |
| TC59, 61, 71 | FR8, FR12 |
| TC65, 66, 68, 69 | FR11, FR17 |
| TC72, 73 | FR12, FR26 |
| TC74 | FR8 |
| *Performance and Latency Tests* | |
| TC75, 85 | FR12 |
| TC76, 77, 90, 91 | FR8 |
| TC78, 79, 81, 86 | FR7, FR26 |
| TC80 | FR7, FR12 |
| TC82-84, 87-89 | FR7 |
| TC92 | FR1, FR1.1 |
| TC93, 96, 100-102 | FR1.1, FR21 |
| TC94, 95, 99 | FR11, FR21 |
| TC97, 103 | FR1 |
| TC98 | FR11, FR17, FR21 |
| TC104 | FR1.1, FR21 |
| *Profile Activity Tests* | |
| TC105-110 | FR21, FR23 |

| | |
|---|---|
| TC111-114 | FR10, FR11, FR17, FR18, FR21 |
| TC115-117 | FR10, FR11, FR17 |
| TC118-126 | FR10, FR18 |
| TC127-132 | FR10, FR11, FR18, FR19, FR22 |
| TC133-136 | FR18, FR21 |
| TC137-143 | FR17, FR18, FR19, FR22 |
| TC144-146 | FR2, FR3 |
| *Exercise Data Tests* | |
| TC147-155, 164-166 | FR5, FR18, FR21, FR27 |
| TC156-163 | FR6, FR18, FR21 |

Table 10 showcases both the functional requirements and their respective test cases.

Table 11: Usability Tests and Corresponding Requirements

| **Test ID** | **Non-Functional Requirement** |
|---|---|
| UT-1 | LFR-AP1 |
| UT-2 | UHR-EOU2, UHR-LRN1 |
| UT-3 | UHR-EOU1, LFR-ST1 |
| UT-4 | UHR-EOU1, UHR-LRN1 |
| UT-5 | UHR-EOU2, UHR-AR1 |
| UT-6 | UHR-UPL1, UHR-AR1 |
| UT-7 | UHR-LRN1, UHR-EOU1 |
| UT-8 | UHR-EOU1, LFR-ST1 |
| UT-9 | UHR-EOU1, LFR-AP1 |
| UT-10 | UHR-EOU1, UHR-UPL1 |

Table 11 maps each of the Usability tests to their respective non-functional requirements.

Table 12: Performance Tests and Corresponding Requirements

| Test ID | Non-Functional Requirement |
|---|---|
| PR-1 | PR-SL1, PR-SL2 |
| PR-2 | PR-SL1 |

Table 12 maps each of the Performance tests to their respective non-functional requirements.

Table 13: Reliability & Fault-Tolerance Tests and Corresponding Requirements

| Test ID | Non-Functional Requirement |
|---|---|
| RFT-1 | PR-RFT1 |
| RFT-2 | PR-RFT2 |

Table 13 maps each of the Reliability and Fault-Tolerance tests to their respective non-functional requirements.

Table 14: Security Tests and Corresponding Requirements

| Test ID | Non-Functional Requirement |
|---|---|
| SR-1 | SR-AR1 |
| SR-2 | SR-AR1 |

Table 14 maps each of the Security tests to their respective non-functional requirements.

Table 15: Maintainability & Support Tests and Corresponding Requirements

| Test ID | Non-Functional Requirement |
|---|---|
| MS-1 | OER-WE1, OER-PR1 |

Table 15 maps each of the Maintainability and Support tests to their respective non-functional requirements.

# 11 Trace to Modules

The following section displays the traceability of each of the test cases to the specific modules they validated, which ultimately showcases the overall verification of our system.

Table 16: Tests for all of the Modules

| **Test ID** | **Module** |
|---|---|
| TC1-8 | M1 (Account Creation) |
| TC9-12 | M2 (Valid User) |
| TC13-22 | M3 (User Account) |
| TC22-30 | M8 (Feedback), M11 (UI) |
| TC31-38 | M6 (Analysis Control Flow), M10 (Draw) |
| TC39-74 | M6 (Analysis Control Flow), M7 (Analyze), M9 (Metrics) |
| TC75-104 | M6 (Analysis Control Flow), M11 (UI) |
| TC105-146 | M5 (User Activity) |
| TC147-166 | M4 (Exercise Data) |
| UT-1, UT-2, UT-3, UT-4, UT-5, UT-6, UT-7, UT-8, UT-9, UT-10, PR-1, PR-2, RFT-1, RFT-2 | M11 (UI) |
| SR-1, SR-2 | M1 (Account Creation), M2 (Valid User) |
| MS-1, MS-2 | M1 (Account Creation), M2 (Valid User), M11 (UI) |

Table 16 maps each of the test cases to their respective modules.

# 12 Code Coverage Metrics

The backend code has a total coverage of 97%, with all modules having coverage above 90%. The analyze.py module has 100% coverage, while draw.py,

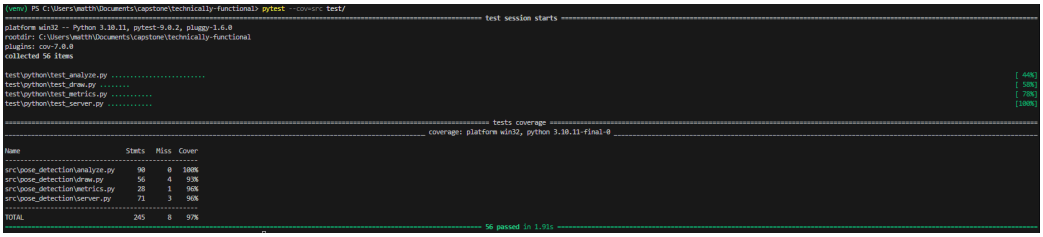


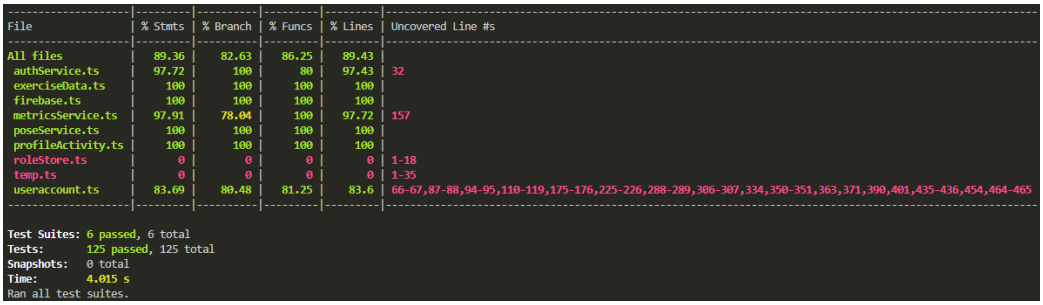

Figure 1: Python code coverage overview showing 97% total coverage across all modules.

| Name | Stmts | Miss | Cover |
|---|---|---|---|
| src\pose_detection\analyze.py | 90 | 0 | 100% |
| src\pose_detection\draw.py | 56 | 4 | 93% |
| src\pose_detection\metrics.py | 28 | 1 | 96% |
| src\pose_detection\server.py | 71 | 3 | 96% |
| TOTAL | 245 | 8 | 97% |

Figure 2: Detailed Python code coverage by module showing analyze.py (100%), draw.py (93%), metrics.py (96%), and server.py (96%).

metrics.py, and server.py have 93%, 96%, and 96% coverage respectively. This indicates that the majority of the backend code is well-tested, with only a small portion of code not covered by tests.

Some areas of improvement would be to fully test each backend module to reach 100% coverage. This would be included in future testing plans but the current coverage points to the reliability of the backend code.



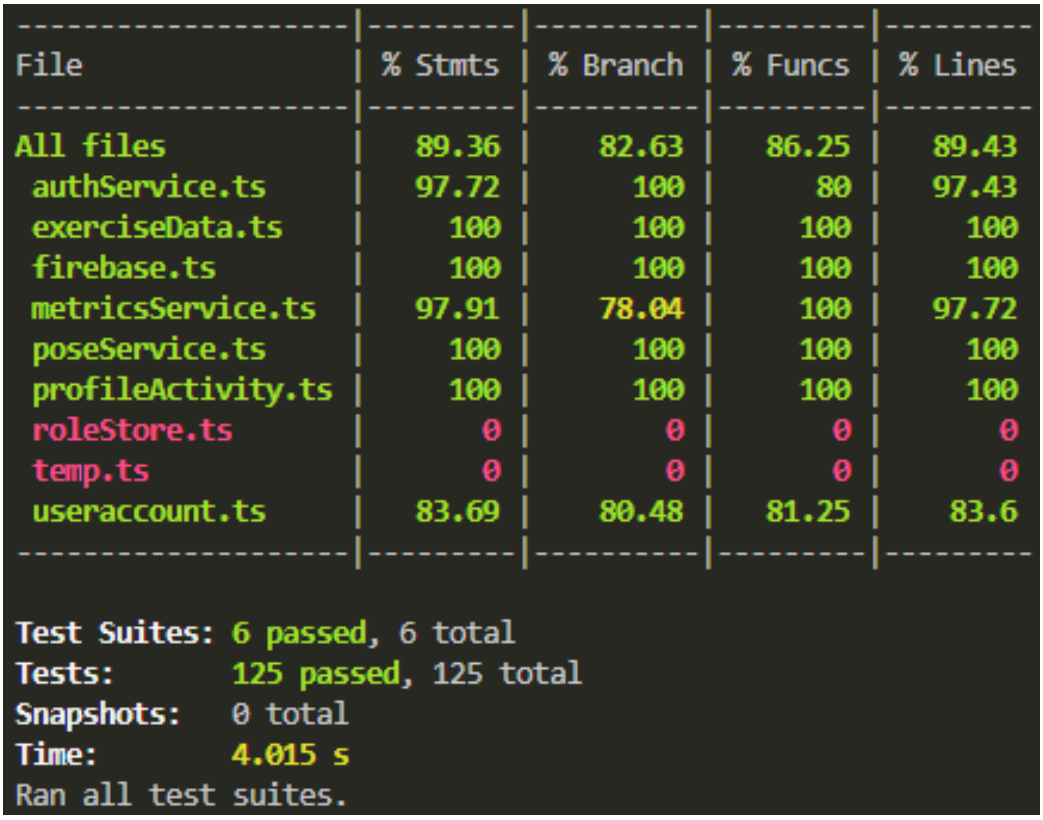
| File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---|---|---|---|---|
| All files | 89.36 | 82.63 | 86.25 | 89.43 | |
| authService.ts | 97.72 | 100 | 80 | 97.43 | 32 |
| exerciseData.ts | 100 | 100 | 100 | 100 | |
| firebase.ts | 100 | 100 | 100 | 100 | |
| metricsService.ts | 97.91 | 78.04 | 100 | 97.72 | 157 |
| poseService.ts | 100 | 100 | 100 | 100 | |
| profileActivity.ts | 100 | 100 | 100 | 100 | |
| roleStore.ts | 0 | 0 | 0 | 0 | 1-18 |
| temp.ts | 0 | 0 | 0 | 0 | 1-35 |
| useraccount.ts | 83.69 | 80.48 | 81.25 | 83.6 | 66-67,87-88,94-95,110-119,175-176,225-226,288-289,306-307,334,350-351,363,371,390,401,435-436,454,464-465 |

Test Suites: 6 passed, 6 total
Tests: 125 passed, 125 total
Snapshots: 0 total
Time: 4.015 s
Ran all test suites.

Figure 3: Detailed TypeScript Code Coverage.

The front end currently tested code has a total coverage of 80%, with major module implementations at or above 80% coverage. UserAccount module currently contains the lowest coverage as several of the functions were implemented originally and are now deprecated and unused, therefor not tested. The high coverage, at or above 97% for most of the modules, shows the testing's extensiveness into statement, branch, and function coverage.

There is room for improvement in the UserAccount module to remove the deprecated functions and increase the coverage. Additionally, there are some minor gaps in testing for some of the modules that could be filled in to reach even higher coverage. Overall, the current coverage indicates that the majority of the front end code is well-tested and will perform as intended.



| File | % Stmts | % Branch | % Funcs | % Lines |
|---|---|---|---|---|
| All files | 89.36 | 82.63 | 86.25 | 89.43 |
| authService.ts | 97.72 | 100 | 80 | 97.43 |
| exerciseData.ts | 100 | 100 | 100 | 100 |
| firebase.ts | 100 | 100 | 100 | 100 |
| metricsService.ts | 97.91 | 78.04 | 100 | 97.72 |
| poseService.ts | 100 | 100 | 100 | 100 |
| profileActivity.ts | 100 | 100 | 100 | 100 |
| roleStore.ts | 0 | 0 | 0 | 0 |
| temp.ts | 0 | 0 | 0 | 0 |
| useraccount.ts | 83.69 | 80.48 | 81.25 | 83.6 |



Figure 4: Detailed TypeScript Code Coverage Close Up.

# 13 Extras

The extras set out for the system consist of both a Design Thinking Document and a Usability Report. The Design Thinking Document aims to explain the process behind, and work done towards, the final design of the system. This document explains the overall flow of processing through the system and the interfacing elements, external libraries utilized, and more. The document flow is iterative, as is the design process, and aims to communicate the entire design process of the system. The Design Thinking Document can be found here. The Usability Report aims to focus on evaluating the system from a user perspective. This report documents usability testing conducted with participants to assess ease of navigation, clarity of instructions, and overall

user experience. It includes observations, user feedback, and analysis of interaction patterns to identify the key areas to foucs on and address. The findings are used to ensure the system meets the non-functional requirements and to guide improvements to the interface for both patients and physiotherapists. The Usability Report can be found here.

# Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

5. Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?

**Vaisnavi:**

- *What went well while writing this deliverable?*

  One thing that went well while writing this deliverable was our ability to collaborate and efficiently split up the tasks and sections of this report. Specifically, in the case of the unit tests and traceability, we decided as a group for each member to derive the unit tests and traceability based on the sections of the app that they had worked on. This aided efficiency and productivity overall, and allowed the report to come together easily.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  One of the biggest pain points I experienced during this deliverable was actually writing the unit tests. Writing the unit tests was actually harder than I had expected because mocking the Firebase database was something I had never done before. Since our authentication and valid user logic uses the Firebase database, we had to mock those database calls instead of using the real database, which took some time to learn and figure out. There was definitely a learning curve, but after looking through some examples and testing different approaches, I was able to get the mocks working correctly. All in all, this pain point allowed me to be able to understand how to test code that depends on external services.

**Maham:**

- *What went well while writing this deliverable?*

  Our team worked really well together. Everyone completed their assigned tasks on time and were available to assist others if needed. We were all communicative, cordial and on schedule.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

At the start, we split off into completing our 'Extras'. I was responsible for the Usability Testing and I had to ensure that there were results to work with. The team members conducted the tests on participants, while I worked on analysis as the results came in. Nearer to the deadline, we realized that the due date for Extras was moved to much later, (I don't think it was in the calendar until very recently), after which all the members hopped on deck to complete their corresponding module tests. Note to self: please double check your dates.

**Eman:**

- *What went well while writing this deliverable?*

  One thing that went well while writing this deliverable was the team's ability to divide the work clearly based on ownership. Since each member had worked on specific parts of the application, it was natural to have each person derive the tests and evaluations for their respective sections. This made the process more efficient and ensured that the tests were written with a solid understanding of the underlying implementation. Communication within the team was consistent throughout, which helped us stay on track and address any blockers quickly.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  One of the main pain points I experienced was setting up and debugging the network connection between the mobile device and the Flask backend during testing. The pose detection and recording functionality depended on the phone being able to reach the server, and resolving the IP configuration and network routing issues took considerable time. Once the connection was established, the full recording flow was tested with the real camera and backend, and the system performed as expected.

**Matthew:**

- *What went well while writing this deliverable?*

One thing that went well while working on this deliverable was our ability to communicate with each other and work together. We were able to make each meeting meaningful by updating the group on what we were working on, if we were stuck and also future deadlines. This allowed us to stay on track and stay on top of the deadlines. We also divided things based on our module implementation and that helped us clearly define the test case implementation and workload for the other documents.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  A big pain point was working on the units test. I have not had much experience with testing and it has been a while since I wrote test cases for school. On top of that the difference in technology that we used academically to now was vastly different so I had to learn how to Mock certain objects and functionalities instead of using an actual database or object. I resolved them by just trying different things and looking at examples online. For example, I had to do a lot of reading on Mock and Patches for testing and the role that they play in testing. In the end I was able to create the unit test and learn a bit on how to test code.

**Cieran:**

- *What went well while writing this deliverable?*
  Learning to use Jest and getting to put into practice all the things I learned from software testing was a great experience. We implemented several tests and explored sample test suite coverage in the class, but actually getting hands on and having to derive tests myself, imagine inputs, edge cases, error scenarios, and problematic inputs was a great learning experience. I think the entire team did a great job at writing tests for their respective modules and a great job on the report overall, completing it in a timely manner and communicating well throughout the process. Specifically, everyone came together for the usability testing to ensure the data we gathered came from as many unique sources and demographics as possible, and could lead to the design of many more implementable improvements both in and out of scope.

- *What pain points did you experience during this deliverable, and how did you resolve them?*
  I was responsible for the traceability section of the document, one which required the input of my teammates to develop and came across some time management issues within my own limitations. Since the traceability section required the input of my teammates, I had to wait for them to complete their respective sections and derive their tests before I could complete the traceability. This was a bit of a bottleneck for me, but I was able to manage it by communicating with my teammates and ensuring that they were aware of the importance of completing their sections in a timely manner so that I could complete the traceability. Additionally, I had to manage my own time effectively to ensure that I could complete the traceability once I had all the necessary information from my teammates. Overall, it was a bit of a challenge, but through communication and time management, I was able to successfully complete the traceability section. Additionally, my own issue, I had very limited knowledge on Jest and TypeScript implementation and testing, so I had to learn how to use Jest on my own beginner implementations in TypeScript, which had a learning curve but I feel has taught me to make better TypeScript and React code overall.

**Group**

- *Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?*

  Parts of this document stemmed from speaking to our clients, specifically by receiving feedback from our peers who acted as proxies for physiotherapy patients. When demonstrating our application, they interacted with the system and gave relevant, useful feedback on the usability of the interface as well as how clear and easy to grasp the real-time exercise feedback was. This mainly had an impact on the sections related to usability testing, interface design, and how feedback is presented to users, especially the requirement for clear instructions and visible/audio cues during exercises. Moreover, the other parts of the document, such as functional requirement evaluation and technical implementation, did not come directly from speaking to our clients. These were based on the requirements defined in our SRS and the

testing done by our core team, since these parts relate more to the development logic and system reliability rather than user interaction.

- *In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)*

  Our VnV Plan deviated from the activities that were actually conducted in a few notable ways. The original plan outlined testing at a higher level of abstraction, targeting broader system functionalities rather than individual module functions. As development progressed and the codebase became more detailed, it became clear that more granular unit tests were necessary. This also coincided with our team further subdividing some of the modules during implementation, which resulted in more detailed tests being required. This led to the addition of the User Account module test suite, which did not exist in the original plan at all. As this module grew in scope and became central to both patient and PT workflows, dedicated testing for functions such as `getUserData()`, `updateUser()`, and `deletePTAccount()` became essential. The usability testing itself proceeded largely as planned, though the participant pool was smaller than initially intended due to scheduling constraints. The structured questionnaire format, pre-session, session, and post-session, was maintained as designed and provided consistent data across all five participants.

- *Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?*

  Our project currently uses GitHub Actions for CI, but as of right now, the workflow is limited and mainly displays the pull request activity and

build checks. With our implemented unit tests for both the frontend and backend, we want to integrate these directly into the CI pipeline. Specifically, the GitHub Actions workflow could be configured to automatically run the Jest test suite for the TypeScript frontend and the Pytest test suite for the Python backend whenever a pull request is opened or updated. In this case, if any test fails, the workflow would essentially fail, which would ultimately prevent changes from being merged until the issue is resolved. With this proposed change, this would make the CI pipeline much more effective for regression testing, as it would automatically verify that new code trying to be merged in does not break existing functionality. It is important to note that it would have been helpful to set this up earlier in the project, and as a result, for future projects, we would aim to do so. Thus, overall for future projects, we would plan to set up CI earlier and integrate our automated test suites (such as Jest and Pytest) so that the tests run automatically on every pull request and prevent merges if any tests fail.